

DA/MA code and updates

Madeleine Watson

With thanks to Kyriacos Skoufaris for the code

DA/MA Code

```
line.config.XTRACK_USE_EXACT_DRIFTS = True

xutil.set_integrator(line)
particle = 'positron'
operation_mode = 'z'
modes = {'z': 45.6e9, # in eV
        'w': 80e9, # in eV
        'h': 120e9, # in eV
        't': 182.5e9 # in eV
        }
energy = modes[operation_mode]

parameters = xutil.log_parameters (None, operation_mode,
                                  particle_type=particle,modes=modes)

# ## Choose a context
context = xo.ContextCpu()
context_tracking = xo.ContextCpu(omp_num_threads='auto')

tw = line.twiss(ene_loss_and_damping=True)
```

1. [set_integrators](#)

Sets integration methods and number of slices for different element types

2. Configures operational mode. In this case for positrons and FCC-ee z-mode

3. [log_parameters](#)

Builds and returns a dictionary of parameters. It initialises particle properties based on the operational mode

4. Initialises a Multi-core CPU context to parallelise particle tracking

5. Runs a Twiss with synchrotron radiation effects

DA/MA Code

'grid_DA': Creates a polar grid where particles are distributed across multiple angles in the transverse plane at increasing radial distances

'grid_MA': Creates a momentum distribution where particles are assigned a specific energy offset to test longitudinal boundaries

6. generate_particle_grid

Creates the particle distribution, based on the study parameters, by building a grid in normalised phase space. The grid is scaled according to the normalised emittances .

```
parameters = {}
line.configure_radiation(model=None)
parameters['study_parameters'] = {
    'ini_cond_type' : 'grid_DA',
    'output_dir' : 'out',
    'number_of_turns' : 2500,
    'number_of_particles' : 1000,
    'inv1': 0, # np.arange(2)+1,
    'inv2': 0, # np.arange(2,2+3)+1,
    'start_element' : 'QD1_R1',
    'ini_cond_nemittance_x': 13000e-6,
    'ini_cond_nemittance_y': 12000e-6,
    'ini_cond_bunch_length': 4.8e-3,
    'ini_cond_energy_spread': 2e-3,
    'ini_cond_energy_offset': None,
    'new_closed_orbit': None,
    'covariance_dispersion_free': False
}

particles, grid_details = xutil.generate_particle_grid (line,
parameters['study_parameters'])
```

DA/MA Code

7. Particles are tracked and turn-by-turn data is taken

8. The particles are sorted to preserve the original indexing, so every data point is mapped to its initial coordinate

9. DA_vs_turns

Computes and visualises DA from the tracking data by identifying the largest stable amplitude before particle loss. It determines where particles are lost as a function of turns and extracts the DA boundary and its minimum value.

```
## Tracking studies
line.configure_radiation(model='mean')

## Change context for multy CPU for tracking
line.discard_tracker()
line.build_tracker(_context=context_tracking)

# Use tracking
line.track(particles, num_turns=parameters['study_parameters']['number_of_turns'],
turn_by_turn_monitor=True, time=True, with_progress=10)
particles.sort(interleave_lost_particles=True)

# plot function
my_xpf.DA_vs_turns(particles, grid_details['num_r_y_points'],
grid_details['num_theta_x_points'], grid_details['x_normalized'],
grid_details['y_normalized'], grid_details['delta_init'])
```

MA_vs_turns

Computes and visualises MA from the tracking data by identifying the maximum transverse amplitude at which particles remain stable before being lost, for each momentum offset. It identifies the stability boundary and produces a plot showing particle survival against momentum deviation

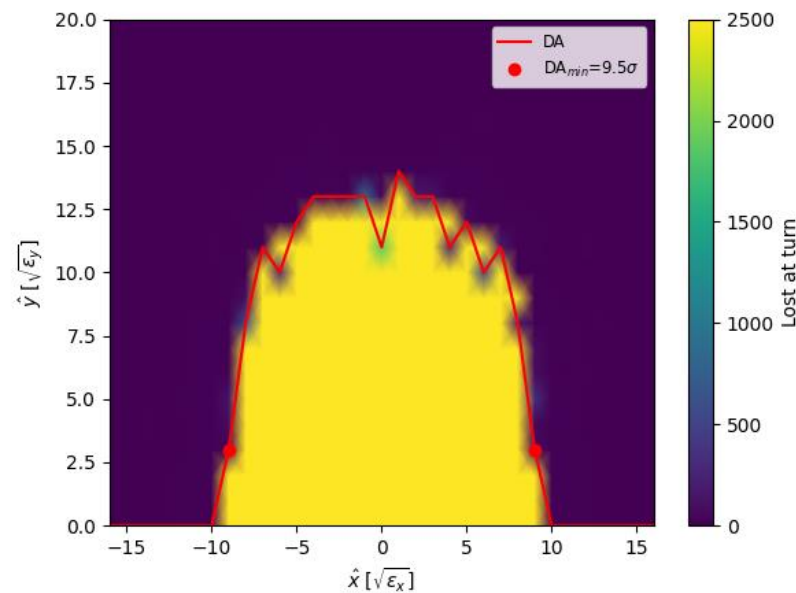
Overview of updates

- Changes to number of slices in DA/MA
- Introduced fringe fields
- Changes made to sextupole configuration
- Frequency map analysis
- Introduced misalignments
- Inserted BPMs and orbit correctors
- Started orbit correction

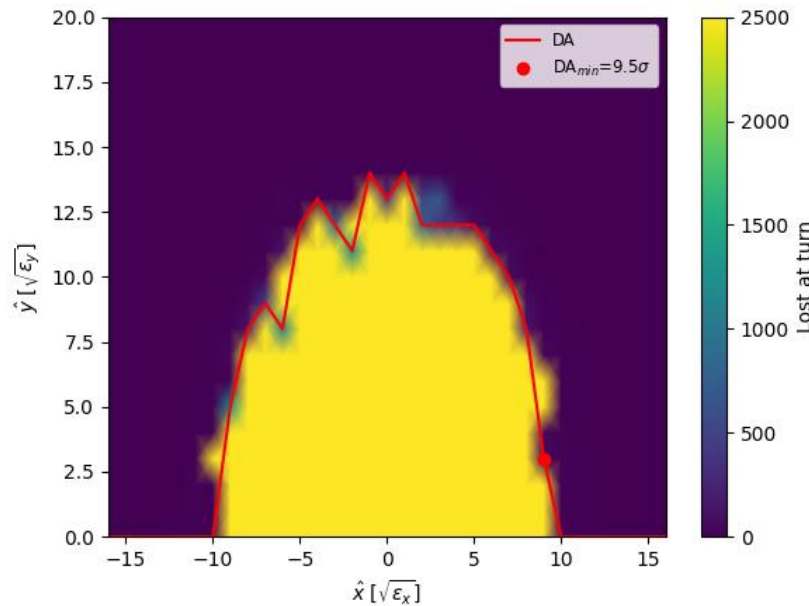
Slicing and fringe fields

- Fringe fields introduced for all magnet types
- Quadrupole slices increased from 3 to 7
- No difference when increasing slices for sextupoles and bends

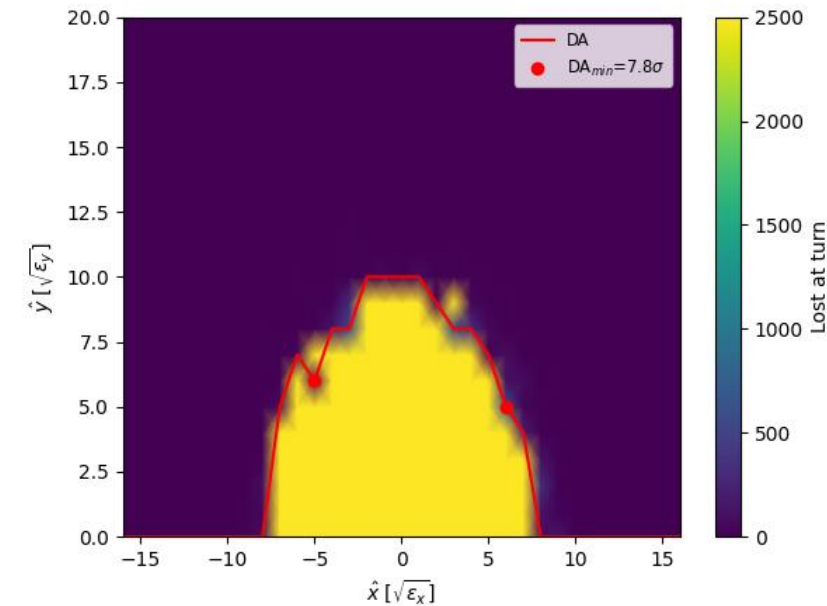
```
pdr.new('Bend', xt.Bend, length='l_bend', angle='hBarc*l_bend', k0_from_h=True,  
        edge_entry_angle='hBarc*l_bend/2', edge_exit_angle='hBarc*l_bend/2',  
        edge_entry_model='full', edge_exit_model='full')  
  
pdr.new('QFarc', xt.Quadrupole, length='l_quad', k1='kQFarc',  
        edge_entry_active=True, edge_exit_active=True)  
  
pdr.new('XF2arc', xt.Sextupole, length='l_sext', k2='k2XF2arc',  
        | edge_entry_active=True, edge_exit_active=True)
```



Original DA plot



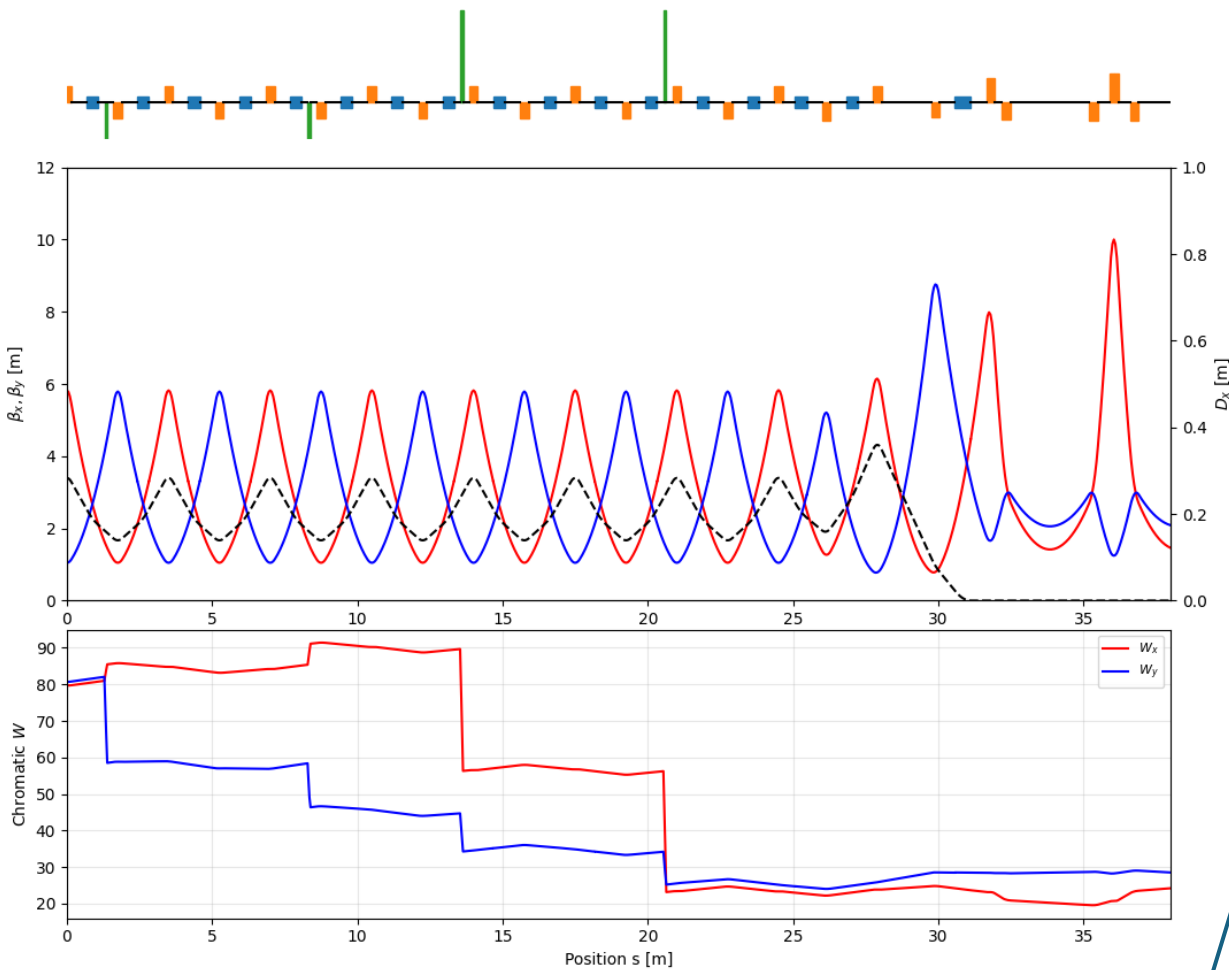
Increased quadrupole slices



With fringe fields

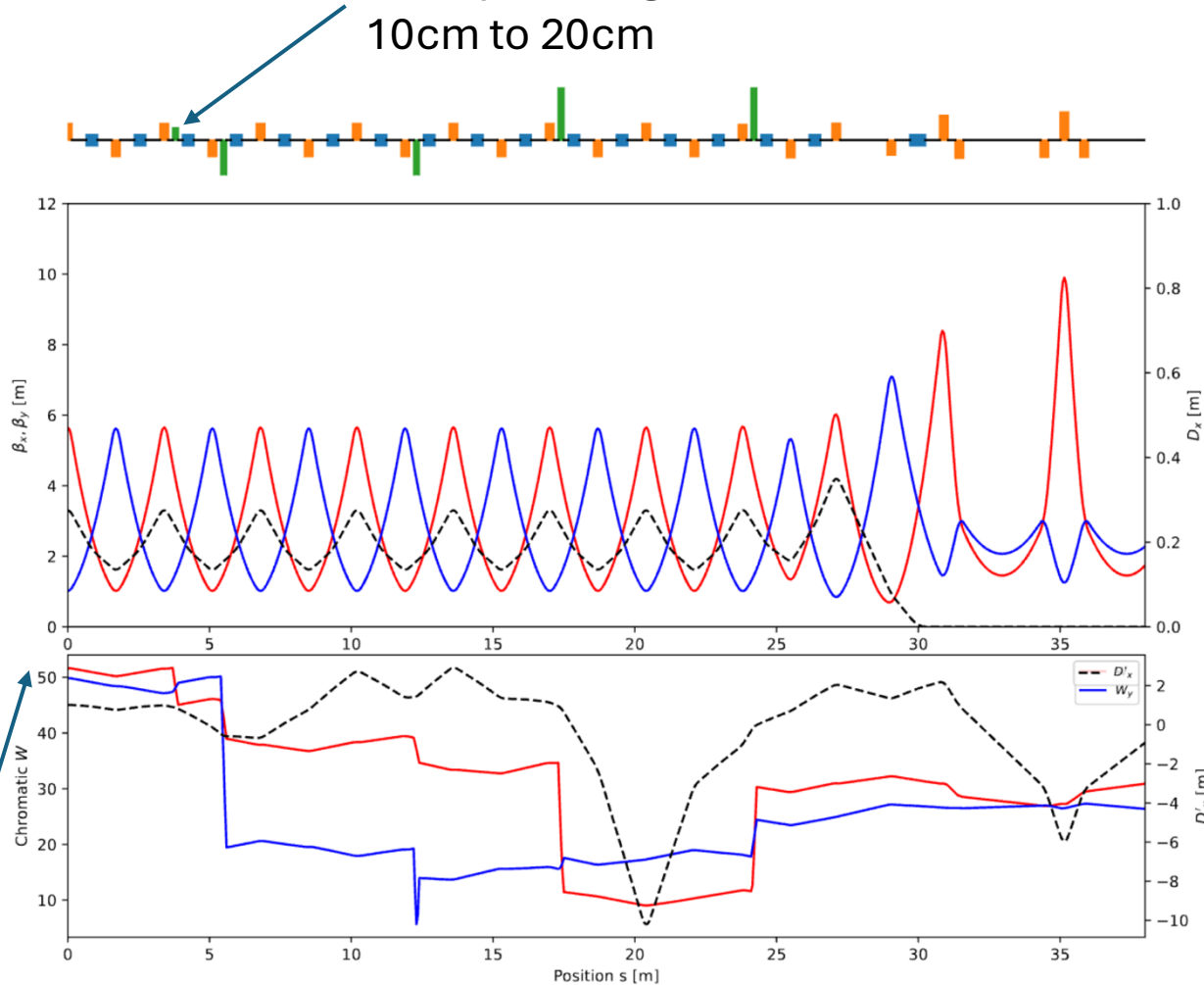
Sextupole configuration change

One focusing sextupole added per half-arc, so three more focusing pairs in the ring
Sextupole length increased from 10cm to 20cm



Previous configuration

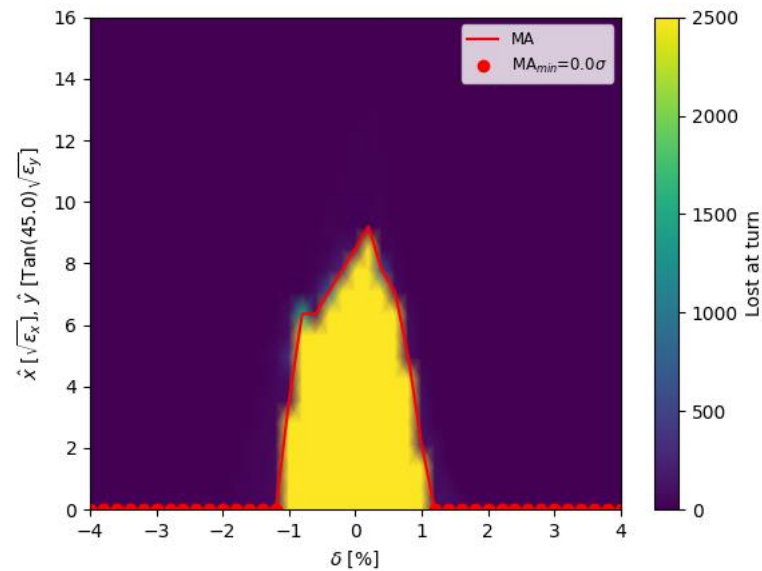
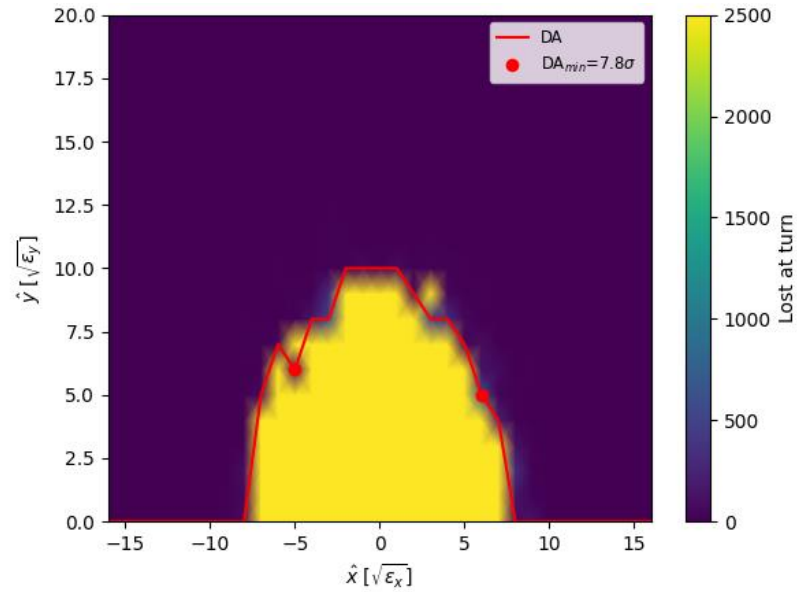
Much lower chromatic functions



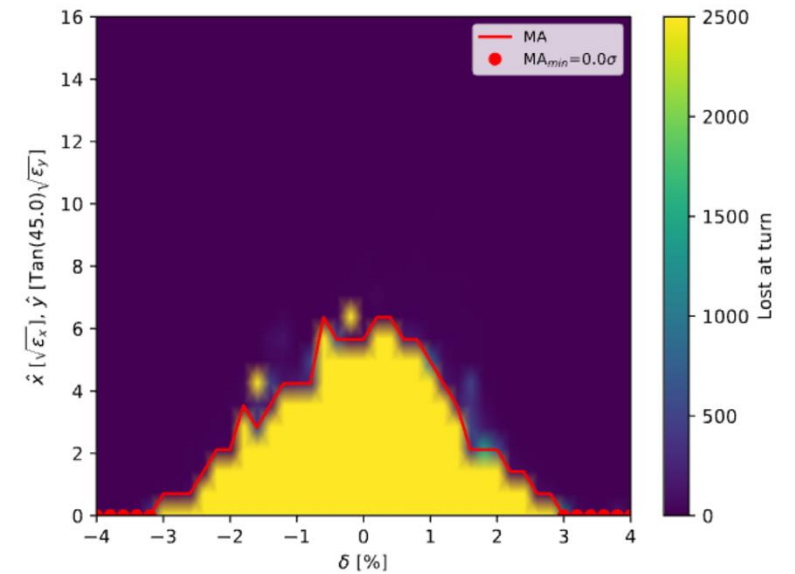
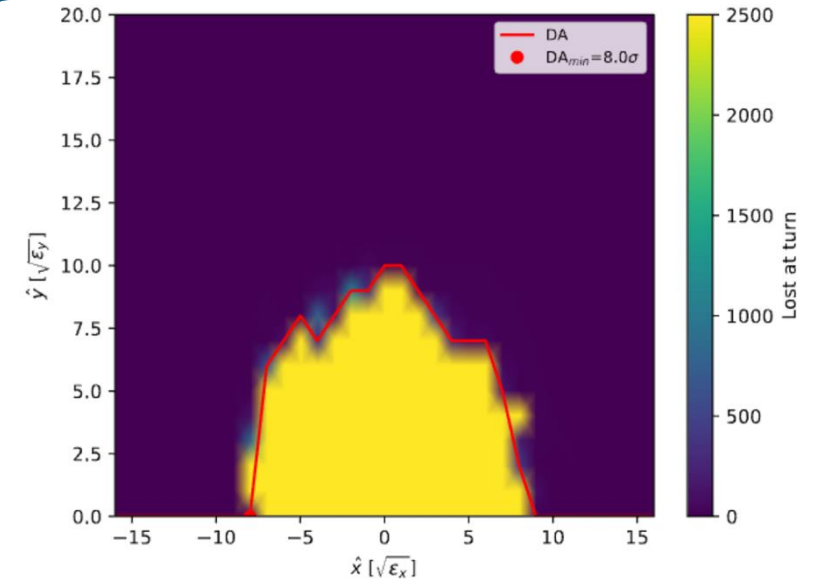
Current configuration

Sextupole configuration change

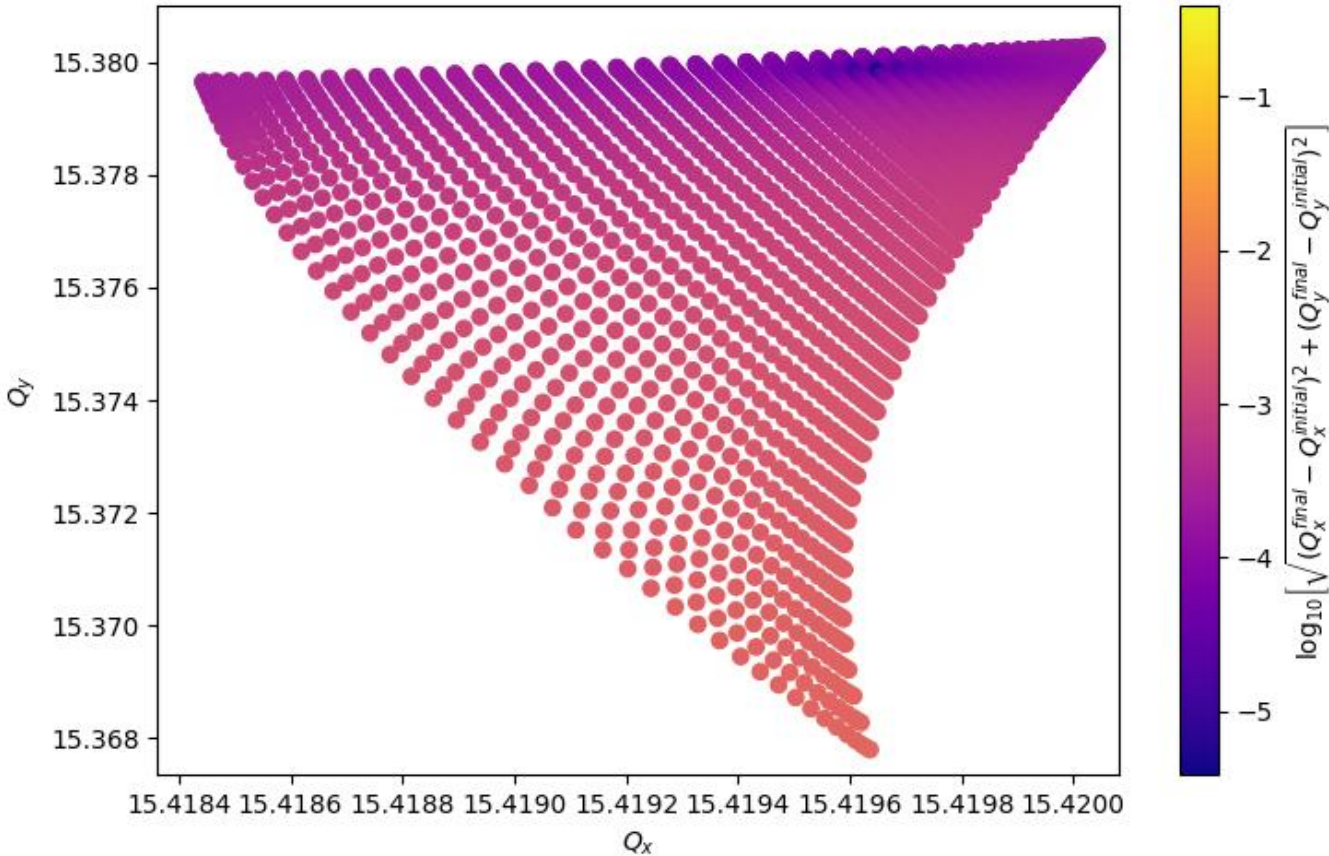
Before
change



After
change



Frequency Map Analysis



```
Qx_start = xutil.nafflib_tune_calculation(line.record_last_track.x[:, :1000],
pq_coordinates=line.record_last_track.px[:, :1000], number_harmonics=1)

Qy_start = xutil.nafflib_tune_calculation(line.record_last_track.y[:, :1000],
pq_coordinates=line.record_last_track.py[:, :1000], number_harmonics=1)

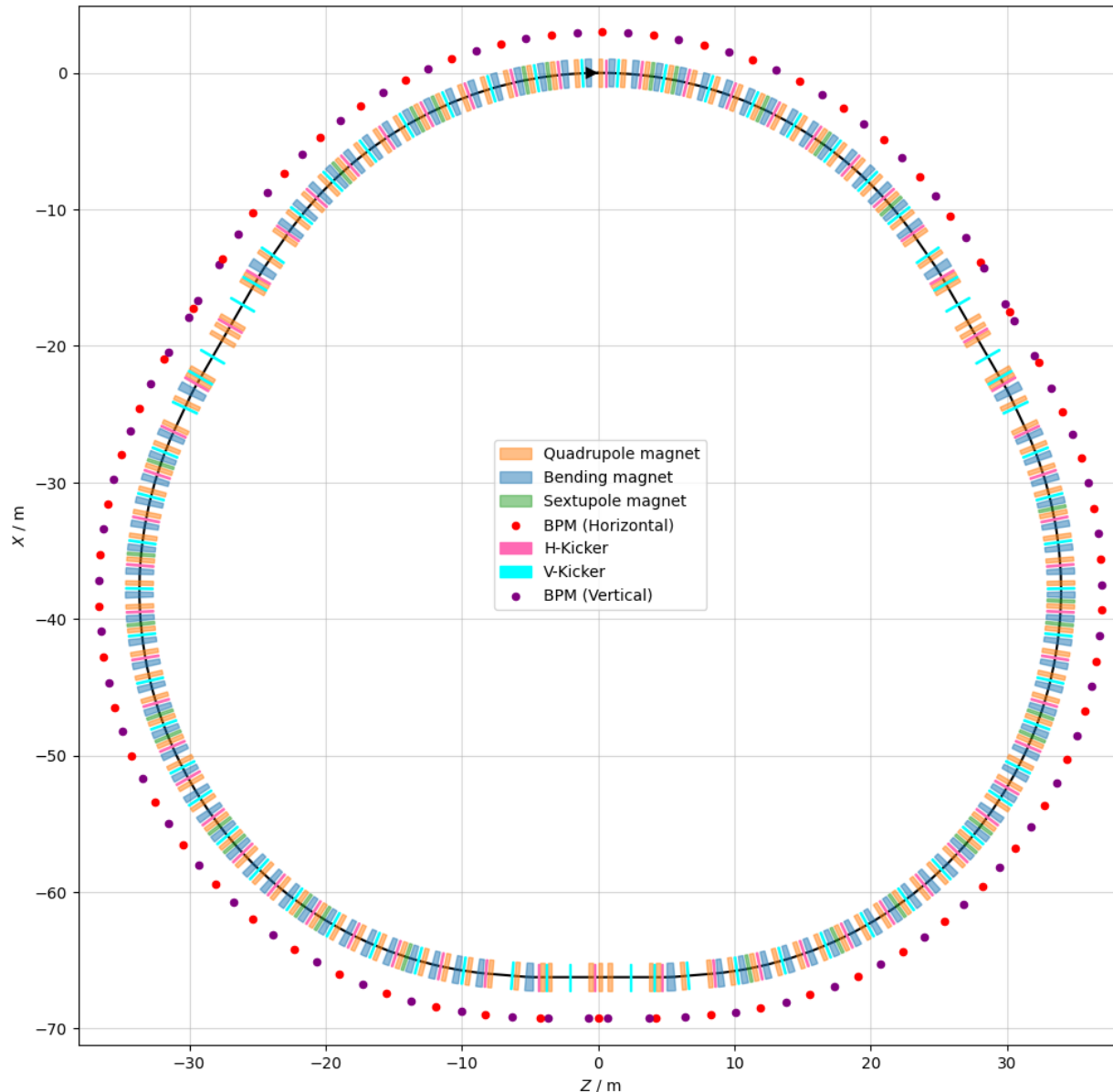
# Last 1000 turns
Qx_end = xutil.nafflib_tune_calculation(line.record_last_track.x[:, 1500:2500],
pq_coordinates=line.record_last_track.px[:, 1500:2500], number_harmonics=1)

Qy_end = xutil.nafflib_tune_calculation(line.record_last_track.y[:, 1500:2500],
pq_coordinates=line.record_last_track.py[:, 1500:2500], number_harmonics=1)

my_xpf.tune_diffusion (Qx_start['Q1']+15, Qx_end['Q1']+15, Qy_start['Q1']+15,
Qy_end['Q1']+15, initial_conditions_x_axis=grid_details['x_normalized'],
initial_conditions_y_axis=grid_details['y_normalized'], xlabel='x [$\sigma$]',
ylabel='y [$\sigma$]', resonance_orders=(1,2,3,4), annotate=True,
delta_value= None)
```

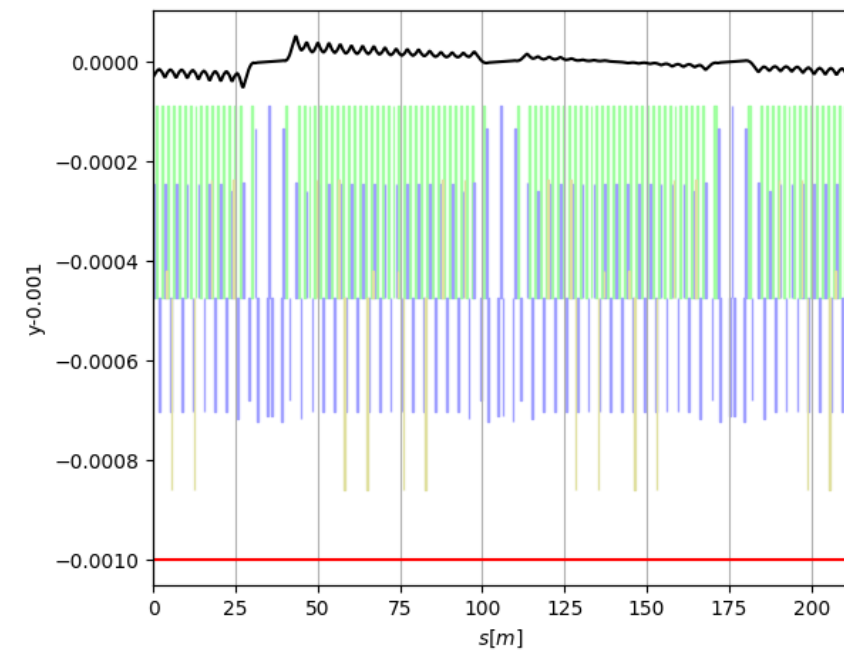
- Tune was calculated for the first and last 1000 turns using NAFF (shown in code above)
- Most particles exhibit low diffusion
- Doesn't cross any resonance lines

Misalignments, BPMs, and correctors

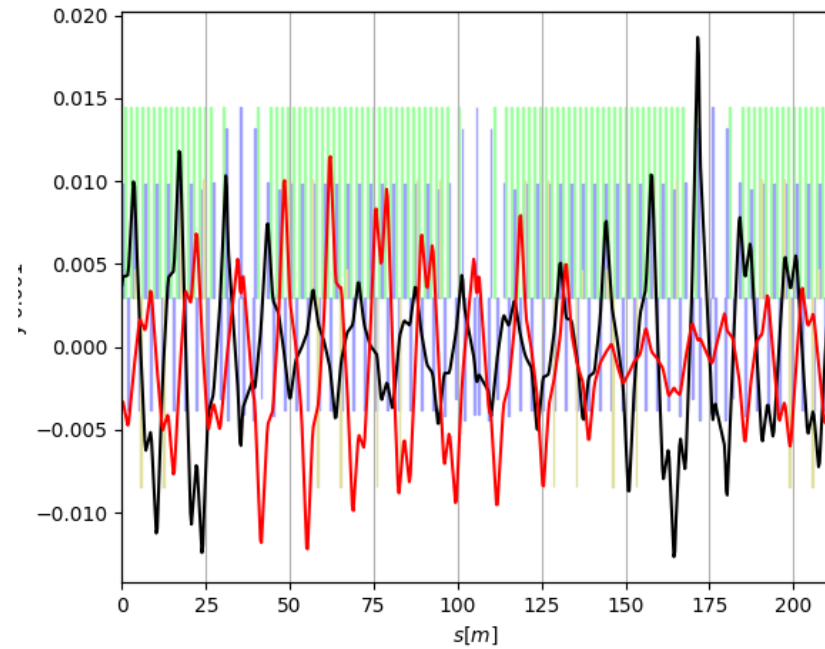


- Misalignments applied in all translational and rotational planes using an error of 0.3mm
- Relative multipole errors of 1/1000 also applied
- Misalignments were applied to all magnet types
- Horizontal BPMs and kickers were inserted around all focusing quadrupoles
- Vertical BPMs and kickers were inserted around all defocusing quadrupoles

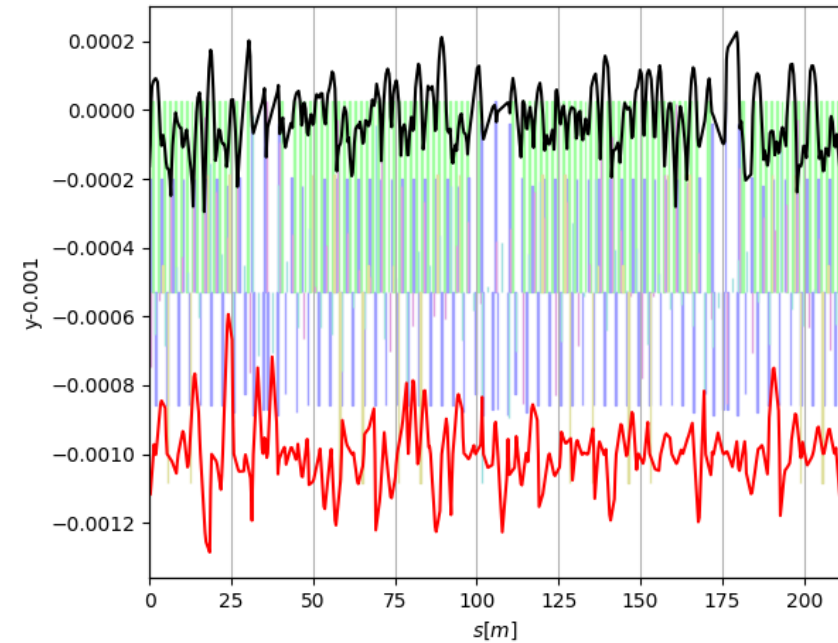
Orbit Correction



Perfect Lattice



With misalignments



Orbit Correction
(MICADO)